Neural Network versus Behavior Based Approach in Simulated Car Racing Game

H. Tang, C. H. Tan, K. C. Tan and A. Tay

***Abstract*—This paper presents examines the design of controllers that is computationally efficient yet demonstrates highly competitive performance for a real time simulated car racing game. Algorithms that require large amount of computational resources are impractical for fast paced and real time games (i.e. racing games, sports simulators, first person shooters and real time strategy games). This paper examines the design of two computationally efficient approaches, neural networks and behaviour based intelligence, in the context of a real time car racing game. Both approaches are optimized using evolutionary strategies. The behaviour based approach was found to obtain a higher fitness value yet being more computationally efficient. The design approaches can also be applied to real-time face animation which involves data-intensive computations.**

I. Introduction

World class game AI exist for many games and are able to defeat good human players [1]. But these are generally restricted to slower paced, turn based, perfect information games [2] such as Deep Blue for International Chess [3] and Chinook for Checkers [4]. Deep Blue compensates for its lack of game reasoning by evaluating board positions millions of times faster than the human player. However, such extreme resource requirements are impractical for faster paced, real time games, i.e. racing games, sports games, flight simulators, first person shooters (FPS) and real time strategy (RTS) because all actions by all players are executed in real time. Any delay in decisions and actions usual meant defeat for the offender. As such, brute force evaluation methods are not feasible. Computationally efficient methods that do not compromise in performance are necessary requirements for the implementation of game AI in such real time games [5]. This paper examines 2 computationally efficient methodologies to design effective controllers for a real time car racing game, Behaviour Based Artificial Intelligence (BBAI) and Neural Networks.

BBAI, popular in the field of robotics, provides some inspiration to address these real time computational requirements. In this methodology [6], intelligence is perceived as a large number of relatively simple and robust modular components. Each of these components work only within a specific set of conditions which it can identify from the environment. BBAI is reactive in nature and operates without search or deliberation, thus is very successful in time critical applications such robotics and interactive virtual reality [7].

Neural networks offer an alternative methodology for implementing game AI in the real time game platform. Although it usually takes a substantial amount of time to train a neural network, a trained network is computationally fast in its application due to it being a string of addition, multiplication and function mapping operations [20] and hence suitable for real time game AI. Neural networks being universal approximators may evolve interesting and effective strategies that are overlooked or unnoticed by behaviour based designs.

Additionally, Computational Intelligence (CI) techniques are used in this paper to simplify the process of designing controller behaviours and optimizing its related parameters. Game agent controllers using evolved neural networks had been successfully applied to games such as racing games [8] [9] [10], soccer simulation [11] [12] [13], role playing games [14], maze chase games [15] [16] and real time strategy games [17] [18] [19] with reasonable performance against an average human player. Design parameters used in behaviour based controllers can also be quickly found and thoroughly exploited.

Two performance metrics, computational efficiency and competitive performance, will be used to evaluate and compare the two approaches.

II. Simulated Car Racing

This section describes the car racing simulator [21] and racing game used as the problem domain in this paper. In this game, 2 players drive their cars in an open arena and earn points by driving through an ordered sequence of waypoints. The player with more points at the end of the stipulated game time wins the game.

The dimension of the competition field is 400 pixels by 400 pixels and is not occupied by any walls or obstacles. As such, vehicles are free to drive even outside of the competition field. The objective of each race is to pass as many waypoints as possible within an allotted time. Two waypoints are visible on the competition field, the current and the next. However, the next waypoint cannot be collected unless the current is passed. That is, the waypoints must be collected in an ordered sequence. Nothing will happen when a car drives through the next waypoint.

H. Tang is with Institute for Infocomm Research, Agency for Science Technology and Research, Singapore 138632 (Email: htang@i2r.a-star.edu.sg).

C. H. Tan, K. C. Tan and A. Tay are with the Department of Electrical & Computer Engineering, National University of Singapore, 4 , Engineering Drive, Singapore 117576 (email: {g0600100, eletankc, eletaya}@nus.edu.sg).

978-1-4244-2754-3/09/$25.00 ©2009 IEEE
Authorized licensed use limited to: Comenius University. Downloaded on March 05,2025 at 09:14:05 UTC from IEEE Xplore. Restrictions apply.

Whenever the current waypoint is passed, the car that passes it gain 1 point, the next waypoint will become the current and a new waypoint will be generated to replace the next.

The vehicles themselves are controlled using digital trigger type controllers like the directional controls found on console game control pads. The 4 distinct on-off control signals, accelerate, decelerate, left and right turn combine to form a total of nine possible controller states, inclusive of a neutral state where no key is asserted. The controller can take inputs either from the keyboard or an AI algorithm. The included physics model is reasonably detailed, allowing for collisions between vehicles as well as side skidding.

When driving towards the current waypoint, care must be taken not to approach it at too high a speed. A driver that is accelerating and steering towards a waypoint may end up overshooting it and end up orbiting around the waypoint.

Tactically, the driver also can try to predict which driver will reach the current waypoint first. Given the situation, the driver can choose to drive faster towards the current waypoint or drive towards the next waypoint and waiting there instead. This way, the drivers loses a point for the current waypoint but wins the next point once it becomes activated and all is square for the race to the next waypoint after that.

Three heuristic controllers with basic driving behaviours used in the car racing simulator [25] are also used in this paper and will be described in the following subsections. The HeuristicSensibleController is also used as the main training opponent in this paper as it represents a well tuned naïve driver that places sufficient selection pressure on the population during training.

A. GreedyController

The GreedyController (GC) is a simple controller that always outputs the acceleration motor command and there is no upper limit for its speed. It will steer towards the next waypoint intuitively depending on whether the angle to the current waypoint is negative or positive.

B. HeuristicSensibleController

The HeuristicSensibleController (HSC) drives directly towards the current waypoint much like the GC but with an upper speed limit of 7 pixels per time step which is a moderate speed. If the instantaneous speed falls below the speed limit, it exerts the accelerate command but if it is above the limit it simply issues the neutral driving command.

C. HeuristicCombinedController

The HeuristicCombinedController (HCC) is a more complex controller when compared to the HSC. Its behaviour will change depending on whether or not its present position is nearer to the current waypoint than the other car. If it is nearer, it behaves identically to the HSC. However, if it is further away, the controller drives towards the next waypoint instead, stopping in the vicinity of the next waypoint. It reduces to the HSC in solo races.

III. Neural Network Controller

An Artificial Neural Network (ANN) [20] is a massively parallel distributed processor made up of simple processing units which has a natural propensity for storing experiential knowledge. A class of ANN employing Multilayer Perceptrons (MLP) represents one of the widely used and effectively machine learning methods currently applied to data classification and function approximation problems. Although it usually takes a substantial amount of time to train a neural network, a trained network is computationally fast in its application due to it being a string of addition, multiplication and function mapping operations.

Neural network models are used to implement the controller because a properly trained neural network can infer an output from a set of observational inputs. This is useful as it avoids the complicated task of analyzing the system and designing driving rules by hand. However, the disadvantage of this approach is that the resultant neural network acts as a black box control unit, making it difficult to make analysis or draw conclusions from the evolved weight values of the neural network. In this situation, the neural network controller is inspected and described visually to quantify its driving behaviours. In this section, a car racing controller constructed solely by neural networks is explored to study its potential as a real time game controller.

The neural networks used are standard multilayer feedforward fully connected perceptrons with 10 inputs, a single hidden layer with 6 hidden nodes and 2 outputs. The inputs are the angle to the other car, the distance to the other car, the orientation of the other car, the angle to the current waypoint, the distance to the current waypoint, the angle to the next waypoint, the distance to the next waypoint, the direction of movement, the orientation of the car, and the speed of the car. Each neuron implements the hyperbolic activation function. At each time step, the observational inputs are fed from the sensor model to the neural network. The outputs are two real number values, one for steering and one for driving. Four additional threshold variables are defined to discretize the outputs to the on-off controllers of the simulator. The variables SteeringLimitLow and SteeringLimitHigh defines the lower and upper threshold of the steering output while the variable DrivingLimitLow and variable DrivingLimitLow defines the upper and lower threshold for the driving output. The pseudo code for discretizing the output of the neural network can be summarized as follows.

```
if (output_steer < SteeringLimitLow)
    steering = 0;
elseif (steering > SteeringLimitHigh)
    steering = 2;
else
    steering = 1;
if (output_drive < DrivingLimitLow)
    driving = 0;
elseif (output_drive > DrivingLimitHigh)
    driving = 2;
else
    driving = 1;
```

Authorized licensed use limited to: Comenius University. Downloaded on March 05,2025 at 09:14:05 UTC from IEEE Xplore. Restrictions apply.

Note that although SteeringLimitLow is logically supposed to be less than SteeringLimitHigh, it is not enforced in the evolution so as to promote discovery of varied strategies. In a similar way, DrivingLimitLow and DrivingLimitHigh are not constrained in any way. For example, in the event that DrivingLimitLow is evolved to be a large positive number, the controller will likely drive the car in reverse most of the time. This representation of the controller is theoretically capable of driving either forwards or in reverse, and also to come to a complete stop. The neural network weights as well as its output threshold variables are trained using evolution strategy.

A. Simulation Settings

A (1+1) Evolution Strategy (ES) [22], population size of 100, running for 200 generations was used as a training method for the neural network controller. The mutation operator was a Gaussian perturbation with a step size set to a fixed value of 0.1 for all variables. The evolution parameters are summarized in Table I. Each individual is evaluated against the HSC, for 5 rounds of competition, followed by 5 rounds of competitive coevolution against another individual from the population. The competitive coevolution was introduced to prevent over training, encourage better generalization and also to promote population diversity [23]. The fitness function was defined as the number of waypoints the individual collected averaged over the 10 rounds of game play. No solo training was used during the training. Elitism was implemented by retaining the best 4 individuals of each generation. Each chromosome for the neural network controller was encoded with a total of 84 real valued variables, 80 for the neural network weights and bias and 4 for output thresholds. In terms of computation time, each run of 200 generations, consisting of 200 × 100 × 10 = 200000 games took less than 10 minutes to complete.

TABLE I EVOLUTION PARAMETERS FOR NEURAL NETWORK AND BEHAVIOUR BASED CONTROLLERS

Parameter	Neural network	Behaviour based
Method	(1+1)	(1+1)
Population size	100	50
Generations	200	200
Mutation type	Gaussian	Gaussian
Mutation probability	1	1
Mutation step size	0.1	0.1

B. Simulation Results

The evolution of the neural network controller is plotted in Fig. 1. It was observed that the neural network controller stagnated in terms of mean score at around 16 points after the first 30 generations. The comparative results, averaged over 500 games, for the neural network controller against the heuristic controllers are shown in Table II. The mean results are given and the standard deviation quoted in parentheses. In solo runs, it was observed that the neural network controller is able to outperform all 3 heuristic controllers with a higher mean score and yet smaller standard deviation. This implied that the neural network controller was a well optimized and consistent driver. In competitive 2 player games against the heuristic controllers, the neural network controller was able to defeat all its heuristic controllers in mean score over 500 games. In particular, the game against the HCC yielded a very high combined end game score of 16.246 + 15.440 = 31.686. This was due to the waypoint predictive nature of the HCC. When the neural network controller was heading towards the current waypoint, there were instances where its opponent gave up the current waypoint and headed towards the next. So when the neural network controller passed the current waypoint, the HCC very quickly passed the newly activated waypoint. This made the game faster paced and led to high end game scores.

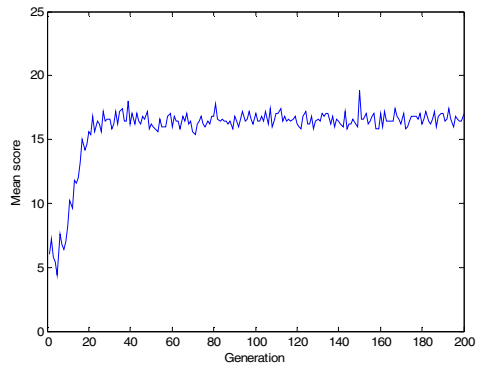

Fig. 1. Training fitness of Neural network controller

TABLE II
RESULTS FOR NEURAL NETWORK CONTROLLER

Controller	Score
Greedy (GC)	12.774 (5.103)
HeuristicSensibleController (HSC)	10.578 (6.601)
HeuristicCombinedController (HCC)	9.284 (6.414)
Neural Network Controller (NN)	20.188 (3.090)
NN - GC	14.692 (2.092) - 11.020 (1.891)
NN - HSC	14.516 (2.240) - 11.996 (2.104)
NN - HCC	16.246 (3.393) - 15.440 (3.189)

IV. BEHAVIOUR BASED CONTROLLER

The behaviour based controller designed is inspired by the behaviour based design methodology. In this section, the Behaviour Based AI (BBAI) methodology will be briefly described followed by detailed discussion on the various components and performance of the behaviour based controller.

In BBAI, intelligence is made up of a large number of modular components which are relative simple and robust. Each of these components work within a specific set of conditions which it is able to observe from the environment. These components are organized into layers in a hierarchy which are able to interact with one another. The constraints here are that no components will have access to another's internal states but it is possible to observe their inputs and outputs. Additionally, a higher layer may subsume a lower layer by affecting its inputs and outputs. This is known as the subsumption architecture [6]. This methodology was powerful because of its simplicity and robustness. Each individual component encodes only a simple behaviour such as moving forward, turning or avoiding objects, thus can be programmed reliably.

In this controller, the basic driving behaviours of the car racing controller such as accelerating, braking and steering are organized at the lowest level. A separate component for

Authorized licensed use limited to: Comenius University. Downloaded on March 05,2025 at 09:14:05 UTC from IEEE Xplore. Restrictions apply.

the prediction of waypoints is placed at a higher level so that it is able to augment the input of the driving layer in order to dictate which waypoint is more advantageous for the car to drive towards. The behaviour based controller consists of 4 main components, illustrated in Fig. 2.

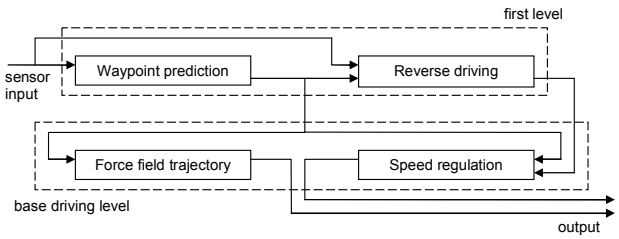

Fig. 2. Overview of behaviour based controller

Force field Trajectory

The first is a trajectory planning mechanism inspired by the interaction between charged particles in space. Potential field methods are widely used in the field of robotics due to its simplicity [24]. Every other particle in the playing game, namely the car belonging to the opponent, the current waypoint and the next waypoint, induces either an attractive or repulsive field in the game field. As any point on the game arena, the controller tries to align the car to the direction of the local induced vector field. As such, the controller car will move along the resultant field lines induced by the interaction of these charged particles. These field lines only indicate the steering path and not the driving speed. The field equation for the particles in the game arena is defined in (1).

$$\vec{E}_i = q_i r^{p_i} \hat{r}\,,\ i = \{other, wp1, wp2\} \tag{1}$$

where other is the opponent vehicle, wp1 is the current waypoint, wp2 is the next waypoint, $\vec{E}_i$ is the field vector induced by the point particle i, q_i is the charge of particle i, r is the distance from the particle with charge q_i and the evaluation point, p_i is the power factor of the distance r and $\hat{r}$ is the unit vector pointing from the particle with charge q_i to the evaluation point. The variables q_i and p_i for the opponent car, the current waypoint and the next waypoint were optimized using evolution strategy. The controller car was considered a point positive charge in calculations in order to evaluate the resultant force exerted on the controller car.

Speed regulation

The force field trajectory component only determines the driving path of the car and not the driving speed. Hence, a speed regulating function, which constitutes the second main component of this controller, is introduced to specify the driving speeds along the steering trajectory. The equation for the speed regulating function is defined in (2).

$$Speed = a \times \tanh\left(b \times r + c\right) + d \tag{2}$$

where r is the distance to the destination and a, b, c and d are parameters characterizing the speed regulation function. The 4 parameters were optimized using evolution strategy. The hyperbolic tangent function was chosen because of its general shape. The tapering of its outputs at high values of r is analogous to the notion that the car should cruise at a constant speed at far distances from its destination (i.e. the cruising speed should not increase indefinitely with distance). Additionally, the steep gradient around the origin is analogous to deceleration when it is near the destination. The values a, b, c and d serves to shape the hyperbolic tangent function to one most desirable for this car racing game. However, there is no constraint that the function need to pass through the origin or that it should be positive or negative.

Reverse driving

A desirable driving feature for this type of point to point race is the ability to drive in reverse. A human player who just started playing the game would very soon realize that if a waypoint is situated at close proximity but directly behind the car, it is faster to simply reverse the car toward it. In implementation, a reverse driving threshold variable is introduced to the behaviour based controller. The angle to the destination was included to determine whether to drive forwards or reverse in a given situation. If the angle is within a given threshold, the speed function will be negated and the controller will reverse the car towards the destination instead.

Waypoint prediction

By observing the state of the game area, the controller predicts which car will reach the current waypoint first. In the event that the opponent is predicted to be faster to the current waypoint, the controller should then direct the car towards the next waypoint instead and vise versa. This predictive module sits on top of the driving layer and is capable of augmenting the inputs to the driving layer.

if (distance (c,wp1) < distance (o,wp1))
    return c1;
elseif
    $\left(\dfrac{distance\,(c,wp1)}{speed\,(c,wp1)} < \dfrac{distance\,(o,wp1)}{speed\,(o,wp1)}\right)$
    return c;
else
    return o;
end

where *distance* (i,j) refers to the Euclidean distance measured between point i and point j, *speed* (i,j) refers to the magnitude of the vector component of the speed of vehicle i along the direction from point i towards point j, c is the car controlled by the controller calling this function, o is the opponent vehicle and wp1 is the current waypoint.

During each time step, the waypoint prediction system determines which vehicle will reach the current waypoint first. If the opponent vehicle will reach first, then the controller will direct both the trajectory planner and the speed regulator towards the next waypoint instead. The waypoint prediction system was designed using simple domain knowledge and reasoning.

A. Simulation Settings

A (1+1) ES, population size of 50, running for 200 generations was used as a training method for the neural

Authorized licensed use limited to: Comenius University. Downloaded on March 05,2025 at 09:14:05 UTC from IEEE Xplore. Restrictions apply.

network controller. As the behaviour based controller has lesser variables in its chromosome, it was trained with a reduced population size of 50, all other conditions remain constant as with the neural network controller. The mutation operator was a Gaussian perturbation with a step size set to a fixed value of 0.1 for all variables. Each individual is evaluated against the HSC for 5 rounds of competition, followed by 5 rounds of competitive coevolution against another individual from the population. The fitness function was defined as the number of waypoints the individual collected averaged over the 10 rounds of game play. Elitism was implemented by retaining the best 4 individuals of each generation. Each chromosome for the behaviour based controller was encoded with a total of 11 real valued variables, 6 from force field component, 4 from speed regulation component and 1 additional variable which encode the threshold for reversing driving.

B. Simulation Results

The force field trajectory component provides some guidelines for the controller to plans its path from the current waypoint to the next. The speed regulating component defines the acceleration, deceleration and stopping behaviour. The reverse threshold decides when the best time to reverse towards a target is. Finally, the waypoint prediction directs the car to the next waypoint if the current waypoint cannot be reached before the opponent. The evolution of the behaviour based controller is plotted in Fig. 3

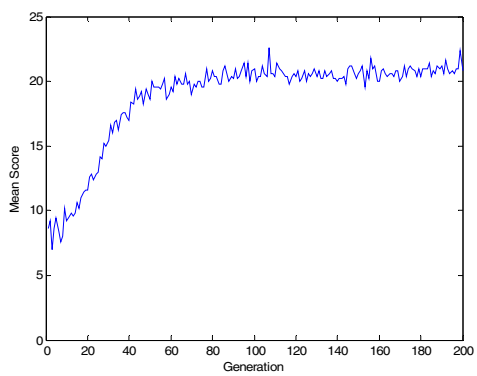

Fig. 3. Training fitness of Behaviour based controller

From the simulation results, it is observed that a driving speed limit of 7 units per second is imposed by the speed regulating function on the controller vehicle. For distances less than 0.2 to the destination, the car switches to rapid deceleration before coming to a halt at the destination point. The negative values of distance r were not used in the actual game as distances were strictly positive. Traveling within these speed limits, the car does not skid and hence does not exhibit any advanced driving techniques that require skidding.

An additional parameter was included to determine whether to drive forwards or in reverse in a given situation. If the angle to the destination is within the threshold stated by the parameter, the speed regulating function will be negated and the controller will reverse the car towards the destination instead. The final evolved value of this parameter is 1.897 radians. This implied that if the waypoint located within a span of 142.5° centred directly behind the car, the controller will drive in reverse towards the destination.

V. Comparative Discussion

The overall performance of the behaviour based controller and its comparison against the neural network controller will be presented in this section. The results of the comparative studies from the neural network controller and the behaviour based controller are summarized in Table III. Part of the result was retrieve from Table I and presented here for better readability. The mean results are given and the standard deviation given in parentheses.

TABLE III Comparative results between Neural network controller and Behaviour based controller

Category	Neural network	Behaviour based
Training best	18.0	22.6
Solo	20.188 (3.090)	22.626 (2.429)
Time taken	76.1 (3.035) seconds	36.1 (0.316) seconds
vs GC	14.692 (2.092) - 11.020 (1.891)	18.630 (2.047) - 10.266 (2.034)
vs HSC	14.516 (2.240) - 11.996 (2.104)	18.726 (2.023) - 11.194 (2.283)
vs HCC	16.246 (3.393) - 15.440 (3.189)	19.388 (4.379) - 15.108 (3.663)
vs each other	13.826 (2.391)	20.324 (1.895)

Since the neural network is a black box controller, its driving behaviour was analyzed visually. While the neural network was randomly initialized, it did evolve into a competent point to point driver that took advantage of the difference in acceleration between driving forward and in reverse. The neural network controller avoided the problem of orbiting a waypoint faced by the heuristic controllers by driving entirely in reverse. The lower acceleration gave it more control in steering and a smaller turning radius that improved its maneuverability around waypoints an easier task. However, the novelty stopped at that.

The neural network controller never learned to decelerate or stop and hence always overshot the target point. The controller did not drive in the opposite direction (i.e. forwards) towards a waypoint. The controller also did not evolve any form of prediction mechanism to decide to approach the next waypoint when the current waypoint was an obvious loss. Therefore, it was concluded that although the neural network representation used here does theoretically allow for the evolution of advanced driving behaviours and strategies, it only exploited the most basic of driving skills in the game and was trapped in a local minima. Perhaps the choice of using a single large neural network to approach this complex racing game was too ambitious. It may be possible to break down the individual aspects of driving and train a separate neural network to learn each part independently or in tandem.

In the solo game, the behaviour based controller obtained a mean score of 22.626 which outperformed the neural network controller's score of 20.188, averaged over 500 trials. At the same time, the behaviour based controller also had a smaller standard deviation implying that it is more competent and also more consistent compared to the neural network controller. Similarly, in 2 player competitions against the 3 heuristic controllers, the behaviour based

Authorized licensed use limited to: Comenius University. Downloaded on March 05,2025 at 09:14:05 UTC from IEEE Xplore. Restrictions apply.

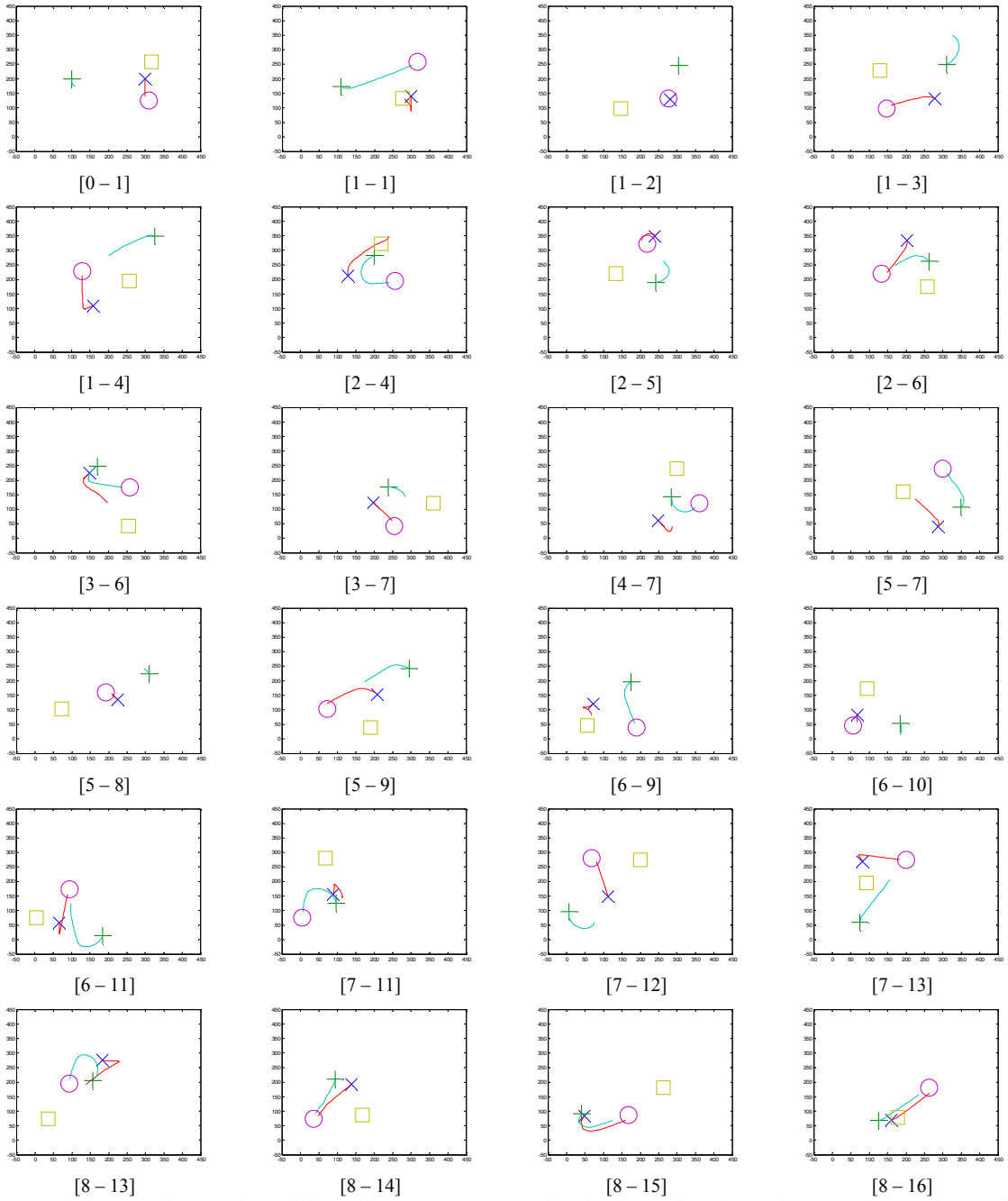

Fig. 4. Point by point diagram of a partial game between Neural network and Behaviour based controller. A circle symbol marks the current waypoint; a square symbol marks the next waypoint; a plus symbol and a cross symbol respectively marks the starting position of the Neural network and the Behaviour based controller when a new waypoint is activated; solid lines marks the paths traced by the respective controllers. Each sub diagram ends when one of the controllers pass the current waypoint, and is annotated by the score of the game just after the current waypoint is passed. The score is read as [Neural network – Behaviour based].

controller was able to achieve larger winning margins as well as higher nominal mean scores. The two controllers were also placed in a direct competition with each other to validate their relative performance with respect to each other. In direct competition, the behaviour based controller scored 20.324 points against the 13.826 points of the neural network controller. A visual inspection of the match up was conducted to further ascertain the reasons for the behaviour based controller's better performance. It was observed that the reverse driving and waypoint prediction components were the main contributors to the success of the behaviour based controller. The reverse driving decision component was able to choose the more time efficient route for the controller to approach its target, as observed in Fig. 4 [6-11] and [7-13]. This could be seen from the sharp angles in the paths traced by the cross symbol (behaviour based controller), while the plus symbol (neural network controller) was seen making large u-turns in Fig. 4 [2-4], [6-11] and [7-12]. The waypoint prediction component enabled the controller to quickly reclaim a point by driving to the next waypoint when the current one cannot be won, as observed in Fig. 4 [2-4], [5-7] and [6-9]. In particular, the

Authorized licensed use limited to: Comenius University. Downloaded on March 05,2025 at 09:14:05 UTC from IEEE Xplore. Restrictions apply.

cross symbol (behaviour based controller) traced a path towards the square symbol (next waypoint) when it predicted a loss for the current waypoint. These features made the driving path traced by the behaviour based controller during the game very fluid and efficient.

A performance indicator raised earlier in this paper was the computational efficiency of the controller. A game AI in a real time driving game such as this do not have the computation resource to evaluate all possible moves at a given game state. Both the neural network and the behaviour based design methodology were considered because of their computationally efficient characteristics. To investigate their comparative time efficiency, both controllers were timed for 10 sets of 5000 solo run trials and the results are also shown in Table II. To establish a common benchmark for comparison, all simulations were conducted on the same computer terminal under the same boot conditions. The neural network controller took 76.1 seconds to complete 5000 solo run trials or 76.1 / 5000 = 0.0152 seconds per trial or 0.0152 / 1000 = 15.2 microseconds per time step while the behaviour based controller took 36.1 seconds for 5000 trials or 0.00722 seconds per trials or 7.22 microseconds per time step. To put the comparison into perspective, in a visual game where the car racing game is graphically simulated on screen, each game typically lasts 60 seconds. Both controllers were computationally efficient but the behaviour based controller was able to outperform the neural network controller while being computationally 2 times faster. The computation gain may not be significant in this simplified simulation as there is no emphasis on graphics and sounds. However, in the context of a commercial game where a large percentage of the CPU cycle is dedicated to rendering graphics and sounds, a computationally efficient game controller becomes desirable. Also, in this simulation, there are only 2 game agents present. In games where there are hundreds of interactive game agents, the savings in computational time becomes significant in ensuring an uninterrupted game presentation.

In expressive facial animation, emotions (e.g., neutral, sad, angry, happy, frustrated) are disconnected in their space and the difficulty is to establish a smooth transition between different emotions to realize emotion mapping. From a recorded training set of speech related emotions, a model can be built by using multilayer feed-forward neural networks to learn the mapping from data. After learning, the neural network model storing the mapping through weights will be able to generate the new emotions in real time. Therefore, neural networks become powerful and competent tools for computer facial animation based on data-driven approaches. Our future focus is to investigate the applications of neural networks in realistic and expressive facial animations.

I. Conclusion

Two design methodologies were considered in this paper, the neural network approach and the behaviour based approach. The evolved behaviour based controller was able to outperform the neural network controller both in terms of competitive scores and computation efficiency. This makes the behaviour based controller a good candidate for graphic intensive games with multiple interacting game agents where efficient usage of computation resources is important for ensuring an uninterrupted game presentation.

References

[1] Lucas, S. M. and Kendall, G., *"Evolutionary Computation and Games"*, IEEE Computational Intelligence Magazine, pp. 10-18, 2006.

[2] Buro, M. and Furtak, T., *"RTS Games as Test-Bed for Real-Time AI Research"*, Proceedings of the Seventh Joint Conf. on Information Science, pp. 481-484, 2003.

[3] IBM Research, Deep Blue, http://www.research.ibm.com/deepblue/, retrieved on 15 May 2007.

[4] Schaeffer, J., *One Jump Ahead: Challenging Human Supremacy in Checkers*, Springer-Verlag, 1997.

[5] Spronck, P., Ponsen, M., Sprinkhuizen-Kuyper, I. and Postma, E., *"Adaptive game AI with dynamic scripting"*, Machine Learning, vol. 63, no. 3, pp. 217-248, 2006.

[6] Brooks, R. A., *"A Robust Layered Control System for a Mobile Robot"*, IEEE Journal of Robotics and Automation, vol. 2, no. 1, pp. 14-23, 1986.

[7] Sengers, P., *"Do the thing right: An architecture for action expression"*, Proceedings of the Second International Conference on Autonomous Agents, pp. 24-31, 1998.

[8] Chaperot, B. and Fyfe, C., *"Advanced artificial intelligence techniques applied to a motocross game"*, Computing and Information Systems, vol. 10, no. 2, pp. 27-31, 2006.

[9] Togelius, J. and Lucas, S. M., *"Arms races and car races"*, Proceedings of Parallel Problem Solving from Nature, 2006.

[10] Togelius J. and Lucas, S. M., *"Evolving robust and specialized car racing skills"*, Proceedings of the IEEE Congress on Evolutionary Computation, 2006.

[11] Nakashima, T., Udo, M. and Ishibuchi, H., *"A fuzzy reinforcement learning for a ball interception problem"*, Lecture Notes in Computer Science, vol. 3020, pp. 559-567, 2004.

[12] Nakashima, T., Takatani, M., Udo, M. And Ishibuchi, H., *"An evolutionary approach for strategy learning in robocup soccer"*, IEEE International Conference on Systems, Man and Cybernetics, vol. 2, pp. 2023-2028, 2004.

[13] Stone, P., Sutton, R. S. and Kuhlmann, G., *"Reinforcement learning for RoboCup-soccer keepaway"*, Adaptive Behaviour, vol. 13, no. 3, pp. 165-188, 2005.

[14] Spronck, P., *"Adaptive Game AI"*, Ph.D. thesis, Maastricht University Press, 2005.

[15] Yannakakis, G. N. and Hallam, J., *"Evolving Opponents for Interesting Interactive Computer Games"*, Proc. of 8th International Conference on the Simulation of Adaptive Behavior, 2004.

[16] Yannakakis, G. N. and Hallam, J., *"A Generic Approach for Generating Interesting Interactive Pac-Man Opponents"*, Proc. of 2005 IEEE Symp. on Computational Intelligence and Games, pp. 94-101, 2005.

[17] Bryant, B. D. and Miikkulainen, R., *"Neuroevolution for Adaptive Teams"*, Proceedings of IEEE Congress on Evolutionary Computation, vol. 3, pp. 2194-2201, 2003.

[18] Miikkulainen, R., Bryant, B. D., Cornelius, R., Karpov, I. V., Stanley, K. O. And Yong, C. H., *"Computational Intelligence in Games"*, Computational Intelligence: Principles and Practice, IEEE Computational Intelligence Society, pp. 155-191, 2006.

[19] Stanley, K. O., Bryant, B. D., Karpov I. and Miikkulainen, R., *"Real-Time Evolution of Neural Networks in the NERO Video Game"*. Proceedings of the Twenty-First National Conference in Artificial Intelligence, pp. 1671-1674, 2006.

[20] Haykins, S., *Neural Networks: A Comprehensive Foundation* New York: MacMillan, 1994.

[21] Togelius, J. and Lucas, S. M., *"Evolving Controllers for Simulated Car Racing"*, Proceedings of IEEE Congress on Evolutionary Computation, vol. 2, pp. 1906-1913, 2005.

[22] Rechenberg, I., *Evolutionsstrategie '94* Stuttgart, Germany: Frommann-Holzboog, 1994.

[23] Goh, C. K., Tan, K. C. and Tay, A., *"A Competitive-Cooperation Coevolutionary Paradigm for Multi-objective Optimization"*, Proc. of

Authorized licensed use limited to: Comenius University. Downloaded on March 05,2025 at 09:14:05 UTC from IEEE Xplore. Restrictions apply.

IEEE International Symposium on Intelligent Control, pp. 255-260, 2007.

[24] Ren, J., McIsaac, K. A., Patel, R. V. and Peters, T. M., *"A Potential Field Model Using Generalized Sigmoid Functions"*, IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics, vol. 37, no. 2, pp. 477-484, 2007.

[25] Togelius, J. and Lucas, S. M., IEEE CEC 2007 Car Racing Competition, http://julian.togelius.com/cec2007competition/, retrieved on 18 November 2007.

Authorized licensed use limited to: Comenius University. Downloaded on March 05,2025 at 09:14:05 UTC from IEEE Xplore. Restrictions apply.